

Capítulo 8

Integración frontend-backend y pruebas

Desarrollo de Software IX · Código 1493 · Módulo III

Propósito del capítulo

Cerrar la teoría del curso conectando React, Express y MongoDB mediante consumo de APIs, contratos de datos, manejo de estados, validación, pruebas funcionales e integración de extremo a extremo.

Tabla de contenido

1. Panorama del capítulo	3
2. El contrato de integración: la API como frontera	3
3. Los tres estados de la interfaz	4
4. fetch, promesas y asincronía	5
5. CORS: por qué el navegador bloquea	5
6. La capa de servicios en el frontend	6
7. La pirámide de pruebas	6
8. Qué prueba cada nivel en una aplicación Full Stack	7
9. Datos de prueba y semillas	9
10. Depuración sistemática	9
11. Observabilidad básica: registros estructurados	10
12. Usabilidad como verificación	10
13. Caso guía: flujo completo de reserva	11
14. Actividades sugeridas	11
15. Rúbrica breve	12
16. Cierre	12

1. Panorama del capítulo

La integración convierte piezas separadas en una aplicación. Un backend con rutas correctas y un frontend con pantallas bien diseñadas todavía no forman un sistema si no comparten contratos de datos, estados, errores y pruebas. Este capítulo estudia la integración como un problema teórico con dos caras: cómo se comunican las capas a través de una frontera explícita —la API— y cómo se verifica, con distintos niveles de prueba, que esa comunicación produce el comportamiento esperado. La pregunta central no es qué herramienta usar, sino por qué cada decisión de integración y de verificación existe.

Idea central

La integración se valida con un flujo completo: una acción del usuario en React debe llegar al backend, modificar o consultar datos en MongoDB y regresar a la interfaz con un resultado comprensible. La calidad de ese circuito no se afirma: se demuestra con pruebas en varios niveles.

2. El contrato de integración: la API como frontera

Frontend y backend son programas distintos, escritos a veces por personas distintas, que evolucionan a ritmos distintos. Lo único que los mantiene coordinados es un acuerdo explícito sobre la frontera que los separa: el contrato de la API. El contrato define qué ruta se llama, con qué método, qué datos se envían, en qué formato, qué respuesta se espera en caso de éxito y qué errores pueden ocurrir. Cuando el contrato no está escrito, cada lado avanza con supuestos implícitos y la integración se rompe al final, que es el momento más costoso para descubrirlo. El formato dominante de intercambio es JSON: texto estructurado, legible por humanos y máquinas, independiente del lenguaje de cada capa. Esa neutralidad es la razón teórica de su éxito: permite que un cliente en JavaScript converse con un servidor en cualquier tecnología sin compartir código.

Parte del contrato	Pregunta que responde	Ejemplo
Ruta y método	¿Dónde se llama y con qué operación?	POST /api/reservas
Cuerpo de la solicitud	¿Qué campos se envían y de qué tipo?	laboratorioid, fecha, bloque
Respuesta exitosa	¿Qué devuelve la API y con qué código?	201 con la reserva creada
Errores posibles	¿Qué casos debe manejar la interfaz?	400 datos inválidos, 409 choque
Autenticación	¿Requiere credenciales?	Authorization: Bearer <token>

2.1 Los códigos de estado como vocabulario compartido

Los códigos de estado HTTP no son un detalle técnico: son el vocabulario semántico del contrato. Un 200 afirma que la operación se completó; un 201, que además se creó un recurso; un 400, que el cliente envió datos inválidos; un 401, que falta identidad; un 403, que la identidad existe pero no tiene permiso; un 404, que el recurso no existe; un 409, que la operación choca con el estado actual del sistema; un 500, que el servidor falló por causas propias. Usar siempre 200 y describir el error solo en el cuerpo destruye este

vocabulario: obliga al cliente a inspeccionar cada respuesta para saber si algo salió bien, e impide que herramientas intermedias —proxies, monitores, pruebas automatizadas— interpreten el resultado sin leer el contenido.

Familia	Significado	Responsabilidad de corregir
2xx	La operación se completó como se esperaba.	Nadie: es el camino correcto.
4xx	El cliente envió algo inválido o no permitido.	El frontend o el usuario.
5xx	El servidor falló al procesar una solicitud válida.	El backend.

2.2 Errores estructurados

Un error también es parte del contrato. Si cada ruta del backend inventa su propio formato de error —a veces un texto plano, a veces un objeto, a veces nada—, el frontend no puede manejarlos de forma uniforme. La solución teórica es el error estructurado: toda respuesta de fallo comparte una misma forma, con un código de estado correcto, un identificador del tipo de error y un mensaje pensado para que la interfaz pueda mostrarlo o traducirlo. El backend nunca debe filtrar detalles internos como stack traces o consultas de base de datos: además de inútiles para el usuario, son información valiosa para un atacante.

```
// Respuesta de error uniforme (409 Conflict)
{
  "error": {
    "codigo": "CHOQUE_HORARIO",
    "mensaje": "El laboratorio ya está reservado en ese bloque.",
    "detalles": { "bloque": "10:30-12:00" }
  }
}
```

3. Los tres estados de la interfaz

Toda vista que consume datos remotos vive una vida en tres tiempos: primero no tiene datos (carga), luego puede haber fracasado (error) o haber recibido respuesta (datos). Esta tríada no es una convención estética sino una consecuencia lógica de la asincronía: entre el momento en que el componente se monta y el momento en que la red responde existe un intervalo que la interfaz debe representar de alguna manera. Ignorar ese intervalo produce pantallas en blanco, datos indefinidos y errores de renderizado. A los tres estados fundamentales se suma un cuarto caso particular del estado con datos: la colección vacía, que merece un mensaje orientador en lugar de una pantalla muda. Diseñar los estados antes de programarlos obliga a pensar la experiencia completa, no solo el camino feliz.

Estado	Qué ve el usuario	Error típico si se ignora
Cargando	Indicador de progreso o esqueleto de la vista.	Pantalla en blanco; doble envío de formularios.
Error	Mensaje accionable con opción de reintento.	Fallo silencioso; el usuario cree que funcionó.
Con datos	Lista, tabla o detalle del recurso.	Acceso a propiedades de un valor aún indefinido.
Vacío (caso especial)	Mensaje orientador: qué hacer para tener datos.	Pantalla muda que parece un error.

Precedencia de los estados en React

Los tres estados se modelan con variables explícitas —bandera de carga, valor de error y el dato en sí— y el renderizado se decide en cascada: si carga, indicador; si error, mensaje; solo entonces, datos. El orden no es arbitrario: refleja la precedencia lógica de los estados y garantiza que el componente nunca intente presentar información que todavía no existe.

4. fetch, promesas y asincronía

La comunicación con el backend es inherentemente asíncrona: la respuesta tarda un tiempo impredecible y el navegador no puede congelarse mientras espera. JavaScript modela esa espera con promesas: objetos que representan un valor futuro que puede resolverse con éxito o rechazarse con un error. La sintaxis `async/await` permite escribir código asíncrono con la forma del código secuencial, sin perder la naturaleza no bloqueante. La función `fetch`, disponible en todos los navegadores modernos, devuelve una promesa con la respuesta HTTP; leer su cuerpo JSON es, a su vez, otra operación asíncrona.

```
async function obtenerReservas() {
  const respuesta = await fetch("/api/reservas", {
    headers: { Authorization: `Bearer ${token}` },
  });
  if (!respuesta.ok) {
    const cuerpo = await respuesta.json();
    throw new Error(cuerpo.error.mensaje);
  }
  return respuesta.json();
}
```

4.1 Errores de red frente a errores HTTP

Un punto teórico que confunde a muchos equipos: `fetch` solo rechaza su promesa cuando la solicitud no pudo completarse —no hay conexión, el servidor no existe, la petición fue bloqueada—. Si el servidor respondió, aunque sea con un 500, la promesa se resuelve con normalidad. Esto significa que existen dos categorías de fallo con orígenes distintos: el error de red, que se captura con `try/catch`, y el error HTTP, que se detecta inspeccionando la propiedad `ok` o el código de estado de la respuesta. Una capa de servicios correcta unifica ambos en un solo modelo de error para que los componentes no tengan que distinguirlos.

Categoría	Cómo se manifiesta en <code>fetch</code>	Ejemplo
Error de red	La promesa se rechaza; lo captura <code>try/catch</code> .	Servidor apagado, sin conexión, CORS bloqueado.
Error HTTP	La promesa se resuelve; <code>response.ok</code> es <code>false</code> .	400 validación, 401 sin token, 500 interno.
Error de formato	El cuerpo no es el JSON esperado.	API devuelve HTML de error en vez de JSON.

5. CORS: por qué el navegador bloquea

Durante el desarrollo, el frontend suele servirse desde un origen (por ejemplo, `localhost:5173`) y el backend desde otro (`localhost:3000`). El navegador aplica la política del mismo origen: por defecto, el código

JavaScript de una página no puede leer respuestas de un origen distinto. Esta restricción no es un capricho: protege al usuario de que un sitio malicioso use su sesión activa para leer datos de otro sitio. CORS (Cross-Origin Resource Sharing) es el mecanismo por el cual el servidor declara, mediante encabezados, qué orígenes externos autoriza. La decisión es siempre del servidor; el frontend no puede 'desactivar' CORS porque quien bloquea es el navegador en defensa del usuario.

- Origen = protocolo + dominio + puerto. Cambiar cualquiera de los tres crea un origen distinto.
- Solicitud simple: GET o POST con tipos de contenido básicos; el navegador la envía y luego decide si entrega la respuesta.
- Solicitud con preflight: si lleva encabezados personalizados (como Authorization) o JSON, el navegador envía antes un OPTIONS preguntando si la operación está permitida.
- El backend Express habilita orígenes específicos con el middleware cors; permitir cualquier origen en producción anula la protección.
- Un error de CORS aparece en la consola del navegador, no en el servidor: el backend sí respondió, pero el navegador se negó a entregar la respuesta al código. Quien entiende que CORS es una política de seguridad del navegador —y no un fallo del código— lo resuelve configurando el servidor en minutos.

6. La capa de servicios en el frontend

Si cada componente React construye sus propias llamadas fetch, el conocimiento del contrato se dispersa: las URLs se repiten, el manejo del token se duplica y un cambio en la API obliga a editar decenas de archivos. El principio de diseño que resuelve esto es la separación de responsabilidades: los componentes se ocupan de la interacción y la presentación; una capa de servicios se ocupa de hablar con la API. El servicio centraliza la URL base, los encabezados comunes, la inserción del token y la normalización de errores, y expone funciones con nombres del dominio —crearReserva, obtenerLaboratorios— en lugar de detalles HTTP. La convención es un servicio por recurso —authService, reservasService, laboratoriosService— del que los componentes importan funciones sin llamar nunca a fetch directamente. El beneficio es teórico y práctico a la vez: el contrato vive en un solo lugar, se prueba en un solo lugar y se cambia en un solo lugar; además, la capa de servicios es el punto natural para sustituir la API real por datos simulados durante las pruebas de componentes.

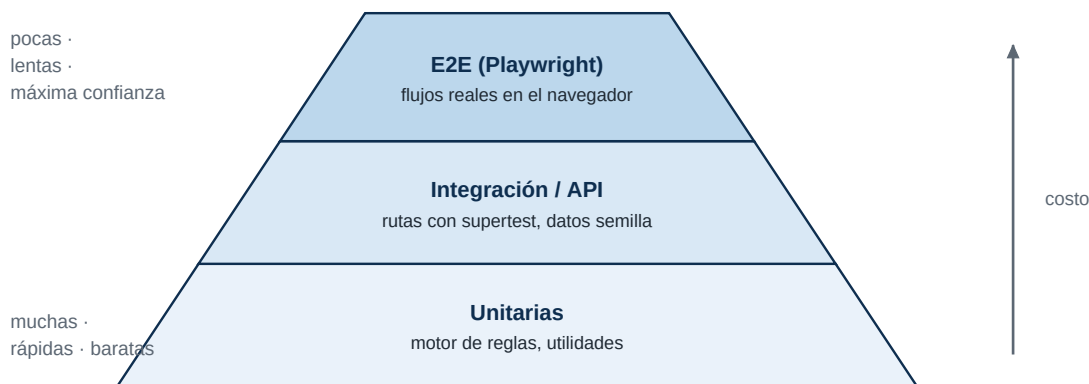
```
// services/reservasService.js
const BASE = "/api/reservas";

export async function crearReserva(datos, token) {
  const r = await fetch(BASE, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify(datos),
  });
  if (!r.ok) throw await construirError(r);
  return r.json();
}
```

7. La pirámide de pruebas

Probar una aplicación Full Stack plantea una pregunta económica: cada nivel de prueba tiene un costo de escritura, un costo de ejecución y un alcance distinto. La pirámide de pruebas es el modelo teórico que ordena esa inversión. En la base están las pruebas unitarias: verifican una función o módulo aislado, se ejecutan en milisegundos y, cuando fallan, señalan el defecto con precisión. En el medio, las pruebas de integración: verifican que varias piezas —rutas, validaciones, base de datos— cooperan correctamente. En la cima, las pruebas end-to-end: ejercitan la aplicación completa como lo haría un usuario, con navegador real. La forma de pirámide expresa la recomendación: muchas pruebas baratas y rápidas en la base, pocas pruebas costosas y lentas en la cima. Invertir la pirámide —confiar todo a pruebas end-to-end— produce suites lentas, frágiles y difíciles de diagnosticar. La pirámide no prohíbe ningún nivel: una prueba end-to-end del flujo crítico vale mucho precisamente porque es la única que demuestra que todo el circuito funciona junto; pero cada regla de negocio individual debe estar cubierta más abajo, donde fallar es barato y diagnosticar es rápido.

Nivel	Qué verifica	Velocidad	Costo de mantener	Precisión del diagnóstico
Unitarias	Una función o módulo aislado.	Milisegundos.	Bajo.	Alta: señala la función exacta.
Integración	Cooperación entre capas (API + datos).	Segundos.	Medio.	Media: señala el flujo.
End-to-end	El sistema completo desde el navegador.	Decenas de segundos.	Alto.	Baja: señala el síntoma.



8. Qué prueba cada nivel en una aplicación Full Stack

Trasladada al stack del curso, la pirámide se concreta así: las pruebas unitarias cubren el motor de lógica —funciones puras de validación, cálculo y reglas de negocio, separadas de Express y de Mongoose—; las pruebas de API cubren los endpoints con sus códigos de estado, validaciones y persistencia; las pruebas end-to-end cubren los flujos críticos del usuario en el navegador. Cada nivel responde una pregunta distinta, y ninguna respuesta sustituye a las otras.

8.1 Pruebas unitarias del motor de lógica

La condición para poder escribir pruebas unitarias baratas es arquitectónica: la lógica de negocio debe existir como funciones puras, sin dependencia del framework ni de la base de datos. Una función que recibe una lista de reservas existentes y una reserva candidata, y devuelve si hay choque de horario, se prueba en milisegundos con casos normales, casos límite y casos inválidos. Si esa misma regla está incrustada dentro del manejador de la ruta Express, probarla exige levantar servidor y base de datos: el diseño determina el costo de la prueba. Esta es la razón profunda por la que separar capas no es burocracia sino economía de verificación.

```
// motor/choques.test.js
test("detecta choque cuando el bloque coincide", () => {
  const existentes = [{ laboratorioId: "L1", fecha: "2026-06-15", bloque: "B2" }];
  const candidata = { laboratorioId: "L1", fecha: "2026-06-15", bloque: "B2" };
  expect(hayChoque(existentes, candidata)).toBe(true);
});

test("no hay choque en bloques distintos", () => {
  const existentes = [{ laboratorioId: "L1", fecha: "2026-06-15", bloque: "B1" }];
  const candidata = { laboratorioId: "L1", fecha: "2026-06-15", bloque: "B2" };
  expect(hayChoque(existentes, candidata)).toBe(false);
});
```

8.2 Pruebas de API

Las pruebas de API verifican el contrato desde afuera, sin navegador: se envía una solicitud HTTP real al backend y se comprueba el código de estado, la forma del cuerpo y el efecto en la base de datos. Pueden ser manuales y documentadas —una colección de Postman con casos exitosos y fallidos para cada endpoint— o automatizadas con herramientas como Supertest, que monta la aplicación Express en memoria y le envía solicitudes desde el propio archivo de prueba. El principio es el mismo en ambos casos: el contrato se prueba antes de conectar el frontend, de modo que cuando la integración falle se sepa de qué lado de la frontera está el defecto. Una suite de API completa cubre el camino feliz, los datos inválidos, el acceso sin credenciales y los conflictos de negocio.

```
// api/reservas.test.js (Supertest)
const request = require("supertest");
const app = require("../app");

test("rechaza reserva sin token con 401", async () => {
  const r = await request(app).post("/api/reservas").send({});
  expect(r.status).toBe(401);
});

test("crea reserva válida con 201", async () => {
  const r = await request(app)
    .post("/api/reservas")
    .set("Authorization", `Bearer ${token}`)
    .send({ laboratorioId, fecha: "2026-06-15", bloque: "B1" });
  expect(r.status).toBe(201);
  expect(r.body.estado).toBe("pendiente");
});
```

8.3 Pruebas end-to-end con Playwright

Las pruebas end-to-end automatizan un navegador real: abren la aplicación, escriben en formularios, hacen clic y verifican lo que el usuario vería. Playwright es la herramienta de referencia del curso. Dos ideas teóricas separan una suite end-to-end sólida de una frágil. Primera: los selectores deben ser accesibles

—localizar elementos por su rol y nombre visible (`getByRole`, `getByLabel`) en lugar de por clases CSS internas—, porque así la prueba verifica lo que el usuario percibe y sobrevive a los cambios de estilo; además, una interfaz fácil de seleccionar por roles es, por construcción, más accesible. Segunda: las esperas deben ser declarativas, no temporales. Esperar 'tres segundos' es apostar; esperar 'a que aparezca el mensaje de confirmación' es verificar. Las pausas fijas son la causa principal del flakiness: pruebas que a veces pasan y a veces fallan sin que el código cambie, erosionando la confianza en toda la suite.

```
// e2e/reserva.spec.js (Playwright)
test("el usuario crea una reserva", async ({ page }) => {
  await page.goto("/reservas/nueva");
  await page.getByLabel("Laboratorio").selectOption("Lab Redes");
  await page.getByLabel("Fecha").fill("2026-06-15");
  await page.getByRole("button", { name: "Reservar" }).click();
  await expect(page.getByText("Reserva creada")).toBeVisible();
});
```

Pocos flujos, pero críticos

En la cima de la pirámide se prueban los recorridos que justifican la existencia del sistema: registro, inicio de sesión, creación del recurso central y el manejo visible de un error de negocio. Una prueba intermitente (flaky) es una deuda: o se corrige la espera, o se corrige la aplicación, pero no se ignora, porque cada falso fallo erosiona la confianza en toda la suite.

9. Datos de prueba y semillas

Toda prueba que toca datos plantea una pregunta previa: ¿en qué estado está el sistema cuando la prueba comienza? Si la respuesta depende de lo que quedó de ejecuciones anteriores, los resultados no son reproducibles. La solución teórica es el dato semilla: un conjunto conocido y versionado de usuarios, recursos y registros que se carga antes de probar y que define un punto de partida idéntico para todos. Las semillas sirven a tres públicos a la vez: a las pruebas automatizadas, que necesitan determinismo; al equipo, que necesita un entorno común para reproducir defectos; y a la demostración del proyecto, que necesita datos creíbles sin depender de escribirlos en vivo.

- Un script de semillas idempotente: ejecutarlo dos veces deja el mismo estado, no duplicados.
- Usuarios de prueba con roles distintos (estudiante, administrador) y credenciales documentadas en el README.
- Cada prueba de API o end-to-end parte de un estado conocido: se siembra antes, se limpia después.
- Las semillas nunca contienen datos personales reales ni secretos.

10. Depuración sistemática

Cuando la integración falla, el instinto es cambiar código hasta que el síntoma desaparezca. La alternativa profesional es un método: reproducir, aislar, hipotetizar y verificar. Reproducir significa encontrar los pasos exactos que provocan el fallo, porque un defecto que no se puede reproducir no se puede estudiar. Aislar significa reducir el espacio de búsqueda por capas: ¿responde el backend correctamente cuando se le llama directo con Postman? Si sí, el defecto está del lado del cliente; si no, del servidor. Hipotetizar significa formular una causa concreta y falsable —'el token no viaja en el encabezado'— en lugar de una vaga —'algo

del login'—. Verificar significa diseñar la observación mínima que confirma o refuta la hipótesis antes de tocar el código. Cada hipótesis refutada también es progreso: reduce el territorio donde puede esconderse el defecto.

- 1 Reproducir: registrar los pasos exactos, los datos usados y el resultado observado frente al esperado.
- 2 Aislar: probar el endpoint directo con Postman para decidir de qué lado de la frontera está el defecto.
- 3 Hipotetizar: proponer una causa específica y comprobable, una a la vez.
- 4 Verificar: observar (consola de red, logs del servidor, base de datos) antes de modificar.
- 5 Corregir y volver a probar el caso original más los casos vecinos.

Síntoma	Hipótesis frecuente	Dónde observar
404	Ruta o prefijo incorrecto.	URL del servicio frontend y rutas Express.
401	Token ausente, mal formado o expirado.	Pestaña de red: encabezado Authorization.
Error CORS en consola	Origen no autorizado por el servidor.	Configuración del middleware cors.
400	El cuerpo no cumple la validación.	Payload enviado frente a esquema del backend.
UI no se actualiza	El estado de React no cambia tras la respuesta.	Componente, hook y flujo del dato.

11. Observabilidad básica: registros estructurados

Depurar exige observar, y solo se puede observar lo que el sistema registra. Un `console.log` suelto con un texto improvisado sirve una vez; un registro estructurado sirve siempre. La idea teórica es tratar cada evento del servidor como un dato: un objeto con marca de tiempo, nivel de severidad, operación, identificadores relevantes y resultado, en lugar de una frase libre. Los registros estructurados se pueden filtrar, contar y correlacionar: encontrar todas las solicitudes fallidas de un usuario, medir cuántos 409 produce la regla de choques, reconstruir la historia de una reserva. El nivel de severidad —info para el flujo normal, warn para lo anómalo recuperable, error para los fallos— permite ajustar cuánto ruido se tolera en cada entorno. Un registro nunca incluye contraseñas, tokens ni datos sensibles.

```
// Registro estructurado en el backend
console.log(JSON.stringify({
  ts: new Date().toISOString(),
  nivel: "info",
  operacion: "crear_reserva",
  usuarioId: req.usuario.id,
  laboratorioId: datos.laboratorioId,
  resultado: "409_choque",
}));
```

12. Usabilidad como verificación

Las pruebas automatizadas demuestran que el sistema hace lo que el equipo programó; la verificación de usabilidad pregunta si lo que el equipo programó es comprensible para una persona. Es una forma de

prueba con método propio: observar a alguien ajeno al equipo intentando completar una tarea real —crear una reserva, recuperarse de un error— sin recibir ayuda. Las heurísticas clásicas de usabilidad dan el marco de evaluación: el sistema debe mostrar en todo momento su estado (¿está cargando?, ¿se guardó?), hablar el idioma del usuario y no el del programador, prevenir errores antes que reportarlos, y ofrecer salidas claras cuando algo falla. La conexión con este capítulo es directa: los estados de carga y error, y los mensajes estructurados del backend, son exactamente el material con el que la interfaz cumple o incumple estas heurísticas.

Heurística	Pregunta de verificación	Ejemplo en el proyecto
Visibilidad del estado	¿El usuario sabe qué está pasando?	Indicador mientras se crea la reserva.
Lenguaje del usuario	¿Los mensajes evitan jerga técnica?	"Ese horario ya está ocupado", no "Error 409".
Prevención de errores	¿La interfaz evita entradas inválidas?	Deshabilitar bloques ya reservados.
Recuperación	¿Hay salida clara cuando algo falla?	Reintentar sin perder lo escrito en el formulario.

13. Caso guía: flujo completo de reserva

El caso guía del curso condensa todo el capítulo en un solo circuito. El usuario inicia sesión, consulta los laboratorios disponibles, completa el formulario de reserva y lo envía. El frontend muestra el estado de carga; el servicio HTTP agrega el token y llama a la API; el backend valida los datos, aplica la regla de choque de horario, persiste el documento en MongoDB y responde 201 con la reserva o 409 con un error estructurado; la interfaz sale del estado de carga y muestra la confirmación o el mensaje accionable, actualizando la lista sin recargar. Cada paso del circuito es verificable en un nivel distinto de la pirámide, y esa correspondencia es la demostración final de la integración.

Paso del flujo	Qué se verifica	Nivel de prueba
Regla de choque de horario	La función pura decide correctamente.	Unitaria.
POST /api/reservas	201 con reserva; 409 con error estructurado; 401 sin token.	API (Postman / Supertest).
Formulario y confirmación	Carga, error y datos visibles para el usuario.	End-to-end (Playwright).
Mensajes y recuperación	Una persona ajena completa la tarea sin ayuda.	Verificación de usabilidad.

14. Actividades sugeridas

- Escribir la tabla de contrato (ruta, método, cuerpo, respuesta, errores) de tres endpoints del proyecto.
- Definir el formato de error estructurado del backend y aplicarlo a todas las rutas.
- Diseñar en papel los estados de carga, error, datos y vacío de una vista antes de programarla.
- Implementar una capa de servicios que unifique errores de red y errores HTTP.
- Probar cada endpoint con Postman en su caso exitoso y al menos dos casos fallidos.
- Escribir pruebas unitarias del motor de lógica con casos normales, límite e inválidos.

- Automatizar con Playwright el flujo crítico del proyecto usando selectores por rol.
- Crear un script de semillas idempotente con usuarios de prueba documentados.
- Documentar la depuración de un defecto real siguiendo reproducir-aislar-hipotetizar-verificar.
- Observar a una persona ajena al equipo completando la tarea principal y registrar las fricciones.

15. Rúbrica breve

Criterio	Excelente	Satisfactorio	Insuficiente
Contrato de integración	Rutas, payloads, códigos y errores estructurados documentados y consistentes.	Contratos parciales o errores sin formato común.	Integración por ensayo y error.
Estados de interfaz	Carga, error, datos y vacío manejados con mensajes útiles.	Estados básicos sin caso vacío ni reintento.	La interfaz se rompe o calla ante fallos.
Pirámide de pruebas	Unitarias del motor, casos de API exitosos y fallidos, flujo crítico end-to-end.	Pruebas en un solo nivel o solo camino feliz.	Solo demostración manual.
Depuración y evidencia	Defectos documentados con método, semillas y registros estructurados.	Evidencia parcial de pruebas.	Sin evidencia reproducible.

16. Cierre

La integración es la evidencia final de una aplicación Full Stack, y las pruebas son la forma en que esa evidencia se vuelve demostrable. Un contrato explícito mantiene coordinadas las capas; los estados de interfaz hacen visible la asincronía; la capa de servicios concentra el conocimiento de la frontera; la pirámide distribuye la inversión de verificación donde rinde más; y la depuración con método convierte los fallos en aprendizaje. El estudiante que domina este capítulo no solo puede mostrar que cada capa de su sistema existe: puede demostrar, con pruebas en varios niveles y evidencia reproducible, que trabajan juntas para resolver el problema definido al inicio del curso.